To implement this with PyTorch, we'll use the `nn` module, which is the higher-level interface for PyTorch, similar to Keras for TensorFlow.

Listing 3.2. Neural network Q function

```
import numpy as np
import torch
from Gridworld import Gridworld
import random
from matplotlib import pylab as plt

l1 = 64
l2 = 150
l3 = 100
l4 = 4

model = torch.nn.Sequential(
    torch.nn.Linear(l1, l2),
    torch.nn.ReLU(),
    torch.nn.Linear(l2, l3),
    torch.nn.ReLU(),
    torch.nn.Linear(l3,l4)
)
loss_fn = torch.nn.MSELoss()
learning_rate = 1e-3
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

gamma = 0.9
epsilon = 1.0
```

So far, all we've done is set up the neural network model, define a loss function and learning rate, set up an optimizer, and define a couple of parameters. If this were a simple classification neural network, we'd almost be done. We'd just need to set up a `for loop` to iteratively run the optimizer to minimize the model error with respect to the data. It's more complicated with reinforcement learning, which is probably why you're reading this book. We covered the main steps well earlier, but let's zoom in a little.

Listing 3.3 implements the main loop of the algorithm. In broad strokes, this is what it does:

1. We set up a `for` loop for the number of epochs.
2. In the loop, we set up a `while` loop (while the game is in progress).